

Orientação a Objetos

Variáveis e Tipos Primitivos

Profa. Alana Neo
Março/2024

Introdução

→ Vamos fazer um programa que imprime uma linha simples.

→ Para mostrar uma linha, podemos fazer:

```
System.out.println("Minha primeira aplicação Java!");
```

→ Você acha que esse código acima, será aceito pelo compilador java?

Introdução

- O código não será aceito pelo compilador java.
- O Java é uma linguagem bastante burocrática, e precisa de mais do que isso para iniciar uma execução.
- O mínimo que precisaríamos escrever é algo como:

```
public class MeuPrograma {  
  
    public static void main(String[] args) {  
  
        System.out.println("Minha primeira aplicação Java!");  
    }  
  
}
```

Introdução

```
class MeuPrograma {  
    public static void main(String[] args) {  
  
        // miolo do programa começa aqui!  
        System.out.println("Minha primeira aplicação Java!!");  
        // fim do miolo do programa  
  
    }  
}
```

- O miolo do programa é o que será executado quando chamamos a máquina virtual.
- Por enquanto, todas as linhas anteriores, onde há a declaração de uma classe e a de um método, não importam para nós nesse momento.

Introdução

```
class MeuPrograma {  
    public static void main(String[] args) {  
  
        // miolo do programa começa aqui!  
        System.out.println("Minha primeira aplicação Java!!");  
        // fim do miolo do programa  
  
    }  
}
```

- Devemos saber que toda aplicação Java começa por um ponto de entrada, e este ponto de entrada é o **método main**.
- Sempre que um exercício for feito, o código que nos importa sempre estará nesse miolo.
- No caso do nosso código, a linha do *System.out.println* faz com que o conteúdo entre aspas seja colocado na tela.

O QUE PODE DAR ERRADO?

- Muitos erros podem ocorrer no momento que você rodar seu primeiro código. Vamos ver alguns deles:

→ Código:

```
class X {  
    public static void main (String[] args) {  
        System.out.println("Falta ponto e vírgula")  
    }  
}
```

O QUE PODE DAR ERRADO?

- Muitos erros podem ocorrer no momento que você rodar seu primeiro código. Vamos ver alguns deles:

- Erro:

```
X.java:4: ';' expected
        }
        ^
1 error

class Main {
    public static void main (String[] args) {
        System.out.println("Falta ponto e vírgula")
    }
}
```

- Esse é o erro de compilação mais comum: aquele onde um ponto e vírgula foi esquecido.
- O compilador é explícito em dizer que a linha 4 está com problemas.

O QUE PODE DAR ERRADO?

- Outros erros de compilação podem ocorrer se você escreveu palavras chaves (as que colocamos em negrito) em maiúsculas, esqueceu de abrir e fechar as { } , etc.
- Durante a execução, outros erros podem aparecer:
 - Se você declarar a classe como X, compilá-la e depois tentar usá-la como x minúsculo (java x), o Java te avisa:

```
Exception in thread "main" java.lang.NoClassDefFoundError:  
X (wrong name: x)
```


O QUE PODE DAR ERRADO?

- Se esquecer de colocar **static** ou o argumento **String [] args** no método **main**:

Exception in thread "main" java.lang.NoSuchMethodError: main

- Por exemplo:

```
class X {  
    public void main (String[] args) {  
        System.out.println("Faltou o static, tente executar!");  
    }  
}
```

O QUE PODE DAR ERRADO?

- Se não colocar o método **main** como **public** :

Main method not public.

- Por exemplo:

```
class X {  
    static void main (String[] args) {  
        System.out.println("Faltou o public");  
    }  
}
```

COMENTÁRIOS EM JAVA

- Para fazer um comentário em java, você pode usar o `//` para comentar até o final da linha, ou então usar o `/* */` para comentar o que estiver entre eles.

```
/* comentário daqui,  
ate aqui */
```

```
// uma linha de comentário sobre a idade  
int idade;
```

Declaração de Variáveis e Tipos Primitivos

- Dentro de um bloco, podemos declarar variáveis e usá-las.
- Em Java, toda variável tem um tipo que não pode ser mudado, uma vez que declarado:

tipoDaVariavel nomeDaVariavel;

- Por exemplo, é possível ter uma idade que guarda um número inteiro:

```
int idade;
```

Declaração de Variáveis e Tipos Primitivos

```
int idade;
```

- Com isso, você declara a variável `idade`, que passa a existir a partir desta linha.
- Ela é do tipo **int**, que guarda um número inteiro.
- A partir daí, você pode usá-la, primeiramente atribuindo valores.
- A linha a seguir é a tradução de: "idade deve valer quinze".

```
idade = 15;
```

Declaração de Variáveis e Tipos Primitivos

- Além de atribuir, você pode utilizar esse valor.
- O código a seguir declara novamente a variável idade com valor 15 e imprime seu valor na saída padrão através da chamada a *System.out.println*.

```
public class Principal {  
  
    public static void main(String[] args) {  
        // declara a idade  
        int idade;  
        idade = 15;  
        // imprime a idade  
        System.out.println(idade);  
    }  
}
```

Declaração de Variáveis e Tipos Primitivos

- Por fim, podemos utilizar o valor de uma variável para algum outro propósito, como alterar ou definir uma segunda variável.
- O código a seguir cria uma variável chamada **idadeNoAnoQueVem** com valor de **idade mais um**.

```
// calcula a idade no ano seguinte  
int idadeNoAnoQueVem;  
idadeNoAnoQueVem = idade + 1;
```

Declaração de Variáveis e Tipos Primitivos

```
public class Principal {  
  
    public static void main(String[] args) {  
        // declara a idade  
        int idade;  
        idade = 15;  
        // calcula a idade no ano seguinte  
        int idadeNoAnoQueVem;  
        idadeNoAnoQueVem = idade + 1;  
        // imprime a idade no ano seguinte  
        System.out.println(idadeNoAnoQueVem);  
    }  
}
```


Declaração de Variáveis e Tipos Primitivos

- No mesmo momento que você declara uma variável, também é possível inicializá-la por praticidade:

```
int idade = 15;
```

```
public class Principal2 {  
  
    public static void main(String[] args) {  
        // declara a idade  
        int idade = 15;  
        // imprime a idade  
        System.out.println(idade);  
    }  
}
```

Declaração de Variáveis e Tipos Primitivos

- Você pode usar os operadores $+$, $-$, $/$ e $*$ para operar com números.

$+$ adição

$-$ subtração

$/$ divisão

$*$ multiplicação

- Além desses operadores básicos, há o operador **% (módulo)** que é o resto de uma divisão inteira.

Declaração de Variáveis e Tipos Primitivos

→ Exemplos:

```
int quatro = 2 + 2;  
int tres = 5 - 2;  
int oito = 4 * 2;  
int dezesesseis = 64 / 4;  
int um = 5 % 2; // 5 dividido por 2 dá 2 e tem resto 1;  
// o operador % pega o resto da divisão inteira
```

- Lembre-se que para rodar esses códigos, você deve colocar esses trechos de código dentro do bloco main, isto é, isso deve ficar no miolo do programa.
- Para imprimir use o **System.out.println**, caso contrário, ao executar a aplicação, nada aparecerá.

Declaração de Variáveis e Tipos Primitivos

- Representar números inteiros é fácil, mas como guardar valores reais, tais como frações de números inteiros e outros?
- Outro tipo de variável muito utilizado é o **double**, que armazena um número com ponto flutuante (e que também pode armazenar um número inteiro).

```
double pi = 3.14;  
double x = 5 * 10;
```

Declaração de Variáveis e Tipos Primitivos

- O tipo ***boolean*** armazena um valor verdadeiro ou falso, e só: nada de números, palavras ou endereços, como em algumas outras linguagens.

```
boolean verdade = true;
```

- ***true*** e ***false*** são palavras reservadas do Java.

Declaração de Variáveis e Tipos Primitivos

- É comum que um ***boolean*** seja determinado através de uma expressão booleana, isto é, um trecho de código que retorna um booleano, como o exemplo:

```
public class IdadeBoolean {  
  
    public static void main(String[] args) {  
        int idade = 30;  
        boolean menorDeIdade = idade < 18;  
  
        System.out.println(menorDeIdade);  
    }  
}
```

Declaração de Variáveis e Tipos Primitivos

- O tipo ***char*** guarda um, apenas **um caractere**.
- Esse caractere deve estar entre **aspas simples**.
- Por exemplo, ela não pode guardar um código como ' ' pois o vazio não é um caractere!
- Variáveis do tipo char são pouco usadas no dia a dia.

```
char letra = 'a';  
System.out.println(letra);
```

Declaração de Variáveis e Tipos Primitivos

- Esses tipos de variáveis são tipos primitivos do Java: o valor que elas guardam são o real conteúdo da variável.
- Quando você utilizar o operador de atribuição = o valor será copiado.

```
int i = 5; // i recebe uma cópia do valor 5
int j = i; // j recebe uma cópia do valor de i
i = i + 1; // i vira 6, j continua 5
System.out.println(i);
```

- Aqui, **i** fica com o valor de 6.
- Mas e **j** ?
- Na segunda linha, **j** está valendo 5.
- Quando **i** passa a valer 6, será que **j** também muda de valor?

Declaração de Variáveis e Tipos Primitivos

```
int i = 5; // i recebe uma cópia do valor 5
int j = i; // j recebe uma cópia do valor de i
i = i + 1; // i vira 6, j continua 5
System.out.println(i);
```

- Quando **i** passa a valer 6, será que **j** também muda de valor?
- **Não, pois o valor de um tipo primitivo sempre é copiado.**
- Apesar da linha 2 fazer **j = i**, a partir desse momento essas variáveis não tem relação nenhuma: o que acontece com uma, não reflete em nada com a outra.

Declaração de Variáveis e Tipos Primitivos

- **Atividade 1 (lembre-se de adicionar comentários para explicar o seu programa):**
 - 1) Faça um programa, para imprimir a sua idade atual e a sua idade no próximo ano.
 - 2) Faça um programa utilizando os operadores +, -, /, * e %. Utilize o *System.out.println*.

Declaração de Variáveis e Tipos Primitivos

→ Atividade 2 Kahoot.it

PIN de jogo: 09406057

Dúvidas



alana.neo@ifms.edu.br